Introduction to Python

Session 13

Third Party Packages
How to create a third-party package ?

Example of a third-party package:
Biopython

Organizing the code

Modules
Packages

Modules

A single Python file containing code

- Any file ending in *.py* is a module
- Divide the python code into manageable parts
- Organize functionality logically
- Reuse code in different programs
- Improve readability and maintenance

Modules

module: file containing Python definitions and statements

- file name: module name
  - The module name can be accessed with the special attribute __name__
- file suffix: .py
- Modules can be executed as scripts:
  - `if __name__ == "__main__":`

Importing modules

- When importing a module with the **import** statement, the module file is looked for in the directories listed in **sys.path** by order**.** When the module is found, it stops looking for it.

```
>>> import sys

>>> print(sys.path)

['', '/Users/xavi/miniconda3/lib/python39.zip',
'/Users/xavi/miniconda3/lib/python3.9',
'/Users/xavi/miniconda3/lib/python3.9/lib-dynload',
'/Users/xavi/miniconda3/lib/python3.9/site-packages']
```

It starts looking in the current working directory

Default third--party package directories

Importing modules

New paths can be added to this list by defining the ENVIRONMENT VARIABLE **PYTHONPATH**

```
$ export PYTHONPATH=$PYTHONPATH:/Users/xavi/pymodules
```

Abandoned in favour or other more portable alternatives.

Create a folder (pymodules) and put it inside pythonpath so that when typing sys.path (previoysly import sys) there will apear the path to the newly folder.

Packages

way of organizing modules

A package is a way of organizing related modules into a hierarchical namespace using dotted names.

`A.B` in this case B will be a submodule inside a package named A

package.module

Submodule named B in a package named A

Packages: Organizing files

https://docs.python.org/3/tutorial/modules.html

in which inside the file will be the code to initialize the package

```
sound/                          Top-level package
      __init__.py a file        Initialize the sound package
      formats/                  Subpackage for file format conversions
              __init__.py
              wavread.py
              wavwrite.py
              aiffread.py
              aiffwrite.py
              auread.py
              auwrite.py
              ...
      effects/                  Subpackage for sound effects
              __init__.py
              echo.py
              surround.py
              reverse.py
              ...
      filters/                  Subpackage for filters
              __init__.py
              equalizer.py
              vocoder.py
              karaoke.py
              ...
```

Packages: Organizing files

https://docs.python.org/3/tutorial/modules.html

```
import sound.effects.echo
```

```
sound.effects.echo.echofilter()
```

```
from sound.effects import echo
```

```
echo.echofilter()
```

```
sound/                          Top-level package
      __init__.py               Initialize the sound package
      formats/                  Subpackage for file format conversions
              __init__.py
              wavread.py
              wavwrite.py
              aiffread.py
              aiffwrite.py
              auread.py
              auwrite.py
              ...
      effects/                  Subpackage for sound effects
              __init__.py
              echo.py
              surround.py
              reverse.py
              ...
      filters/                  Subpackage for filters
              __init__.py
              equalizer.py
              vocoder.py
```

Packages: Organizing files

https://docs.python.org/3/tutorial/modules.html

```
sound/                          Top-level package
      __init__.py               Initialize the sound package
      formats/                  Subpackage for file format conversions
              __init__.py
              wavread.py
              wavwrite.py
              aiffread.py
              aiffwrite.py
              auread.py
              auwrite.py
              ...
      effects/
              __init__.py
              echo.py
              surround.py
              reverse.py
              ...
      filters/
              __init__.py
              equalizer.py
              vocoder.py
              karaoke.py
              ...
```

```
from sound.effects import echo
```

```
from . import echo
```

importing the modue echo from the same place we are

```
from sound.formats import waveread
```

```
from .. import formats
```

impotring something from the parent directory

```
from ..filters import equalizer
```

Relative imports are based on the name of the current module!

Packages: Naming convention

https://www.python.org/dev/peps/pep-0008/#naming-conventions

Modules: should have **short, all-lowercase names**. Underscores can be used in the module name if it improves readability.

Python packages: should also have **short, all-lowercase names**, although the use of underscores is discouraged.

Style Guide for Python Code

https://www.python.org/dev/peps/pep-0008

PEP 8 – Style Guide for Python Code

Author: Guido van Rossum <guido at python.org>, Barry Warsaw <barry at python.org>, Nick Coghlan <ncoghlan at gmail.com>

Status: Active

Type: Process

Created: 05-Jul-2001

Post-History: 05-Jul-2001, 01-Aug-2013

▸ **Table of Contents**

Introduction

This document gives coding conventions for the Python code comprising the standard library in the main Python distribution. Please see the companion informational PEP describing style guidelines for the C code in the C implementation of Python.

- **Real-world modeling.**
  - Classes, objects, behaviours
    - http://www.uml-diagrams.org/class-diagrams-examples.html

- **High-code reusability**
  Don't reinvent the wheel !

- **Reliability and Flexibility**

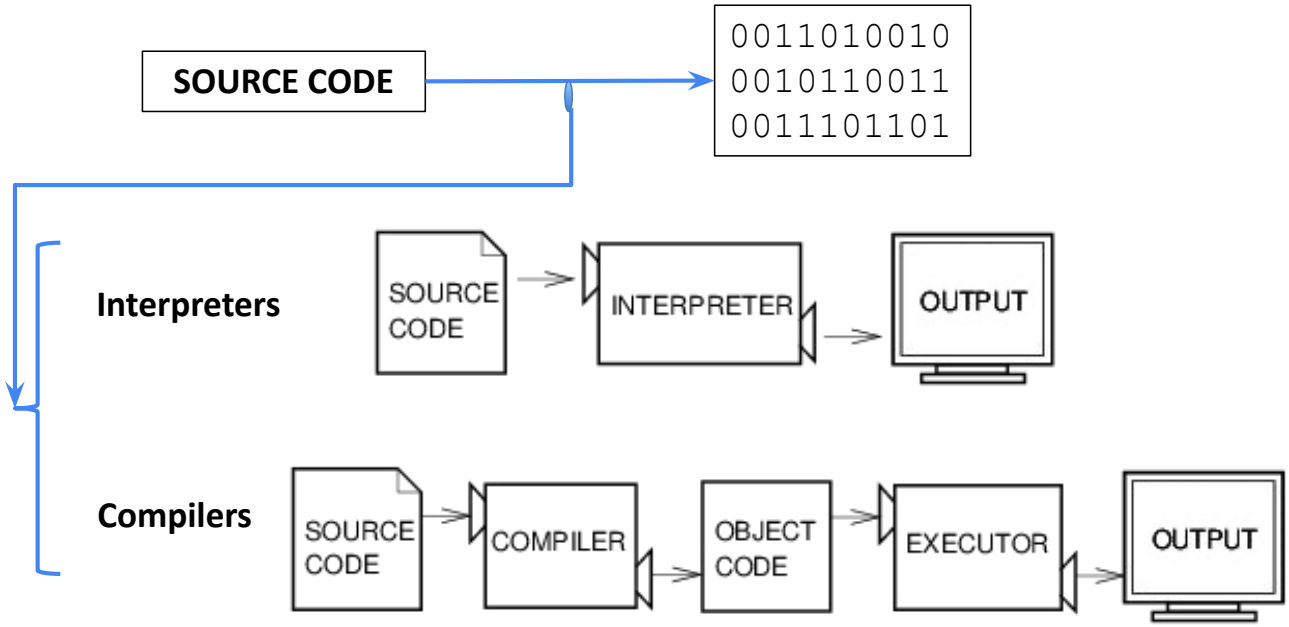
- **Reduced maintenance**

- **But…** sometimes the classes do not match all our needs…
  - **Use inheritance**
    - To add new attributes, new methods,…

Compiled vs. interpreted

Program

How computers understand programs?



http://www.greenteapress.com/thinkpython/html/thinkpython002.html

Python allows to create code in C++

- Performance of a compiled program
- Need to compile when installing the package
  - Automatically managed while installing with setuptools and setup.py

Cython

https://cython.org/

wITH CYTHON LIBRARY
we can add c++ code in our python script

This is not the aim of the Introduction to Python subject, but is good to know it exists...

upf.

Organizing a package

setuptools

how we organize our python package?
We need a way to tell python how to install it. For that a setup.py file is created.

`setup.py`

```
from setuptools import setup, find_packages

setup(
    name='ProjectName',
    version='1.0',
    description='Description of my project',
    author='Xavi Jalencas',
    author_email='xavijalencas@python.org',
    url='http://www.url_of_my_project.org',
    packages=find_packages(),
)
```

Additional arguments: check for necessary third-party packages, need for a specific version of the python interpreter...

Organizing a package

pyproject.toml

more or less same information as setup.py. In this case, we have some dependiencies that our code needs. We can also specify the specific version we need.

```
[project]
name = "ProjectName"
version = "1.0"
description = "Description of my project"
authors = [{name="Xavi Jalencas",
email="xavijalencas@python.org"}]
dependencies = ["requests", "numpy"]

[build-system]
requires = ["setuptools>=61.0"]
build-backend = "setuptools.build_meta"
```

Organizing a package


```
myproject/        main folder
│
├── myproject/
│   ├── __init__.py   code to initalizze the data
│   ├── data.py
│   ├── utils.py
│   └── analysis.py
│
├── tests/   folder for tests
├── pyproject.toml   or the setup.py, better use the .toml one
└── README.md
```

EXAMPLE TO HAVE IN OUR PROJECT!

If we have ML approach maybe another folder named models/

Organizing a package

python provides some ways to install the packages:

To install the package in the computer (installed in the default python site-packages directory):

```
pip install mypackage-0.1.0.tar.gz
```

Create **source distribution files** with all the necessary package files defined.

```
python -m build --sdist
```

Unix: tar.gz file named project_name-version.tar.gz
Windows: Zip file

pip is just ot install python lirbaries
conda is a package manager (like python pakcages), but also C, R, mysql... and different environments.

Conda

Open-source **package** manager and **environment** manager

Python and **non-Python** packages (C libraries, R, etc.)

Popular in data science, machine learning, and scientific computing

Project SBI-PYT

- Install required third-party packages. packages that are not in the standard python library
- Prepare the structure of the program:
  - Package directory structure
  - setup.py or pyproject.toml provide at least!!!!!!!

    if ew use conda, provide a .ymal file describining the enviornment and everyhting to install there!!!!!!!!!
- Installable with all dependencies (pip or conda yaml file)

provide always a .toml file and if use conda, also add .ymal

do not mix, pip or conda. conda deals better with dependencies, checking if all versions match

Example of a third-party package

better install it via conda

conda install conda-forge::biopython

conda forge is the channel

Biopython

http://biopython.org/

biopython

What biopython offers?

- **Parse** bioinformatics files into Python utilizable data **structures** (blast, clustalw, FASTA, Swissprot, SCOP, Pubmed...)
- Code to deal with popular **on-line** bioinformatics destinations such as NCBI or ExPASy
- **Interfaces** to common bioinformatics programs (standalone blast, clustalw,...)
- Common operations on sequences, such as translation, transcription and weight calculations
- ...

Example. Working with sequences:

```
>>> from Bio.Seq import Seq

>>> my_seq = Seq("AGTACACTGGT")
>>> my_seq
Seq('AGTACACTGGT')

>>> my_seq[0]
A

>>> my_seq.count("GT")
2

>>> len(my_seq)
11
```

The Seq class has implemented the special methods `__len__`, `__getitem__`, …

Example. Working with sequences:

```
>>> my_seq + Seq('ATGCGG')
Seq('AGTACACTGGTATGCGG')

>>> set([my_seq, Seq('AGTACACTGGT')])
{Seq('AGTACACTGGT')}

>>> my_seq.transcribe()
Seq('AGUACACUGGU')

>>> my_seq.translate()
Seq('STL')

>>> my_seq.complement()
Seq('TCATGTGACCA')
```

concatenate sequences!

The Seq class has implemented the special method `__add__`, `__hash__`

Example. Working with mutable sequences:

```
>>> from Bio.Seq import MutableSeq

>>> mutable_seq = MutableSeq("GCCATTGTAATGGGCCGCTGAAAG")
MutableSeq('GCCATTGTAATGGGCCGCTGAAAG')

>>> mutable_seq.remove("T")
>>> mutable_seq
MutableSeq('GCCATGTAATGGGCCGCTGAAAG')

>>> mutable_seq.reverse()
>>> mutable_seq
MutableSeq('GAAAGTCGCCGGGTAATGTTACCG')
```

Example. SeqRecord Class

create a seqrecord instance using a seq object

now we have an object with different things in it

```
>>> from Bio.SeqRecord import SeqRecord

>>> record = SeqRecord(
                        (my_seq + "G").translate() ,
                        name="generic protein",
                        id="PROTEIN 1",
                        description="This is protein 1")

>>> record
SeqRecord(seq=Seq('STLV'), id='PROTEIN 1', name='generic
protein', description='This is protein 1', dbxrefs=[])

>>> print(record.format("fasta"))
>PROTEIN 1 This is protein 1
STLV
```

Example. Download a SwissProt file from ExPASy

db for diff class of proteins

```
from Bio import ExPASy
from Bio import SeqIO
handle = ExPASy.get_sprot_raw("O23729")
seq_record = SeqIO.read(handle, "swiss")
handle.close()
print(seq_record.id)
print(seq_record.name)
print(seq_record.description)
print(repr(seq_record.seq))
print("Length %i" % len(seq_record))
print(seq_record.annotations["keywords"])
```

This gets the protein information from the web and creates a SeqRecord object instance

Example. Reading a fasta file

```
>>> from Bio import SeqIO

>>> for record in SeqIO.parse("example_fasta_file.fa”,
                              "fasta"):
          print(record.id)
Q8WZ42
Q5S007
P00533
A2ASS6
P10721
P21802
P31749
P11362
P06213
P09619
P35968
...
```

Returns a generator Object
In each iteration, a SeqRecord object instance is returned

Example. Reading other formats

```
>>> from Bio import SeqIO

>>> for record in SeqIO.parse("ls_orchid.gbk",
                              "genbank"):
          print(record.id)
          print(len(record))
```

Returns a generator Object
In each iteration, a SeqRecord object instance is returned

Example. Retrieve data from Entrez database

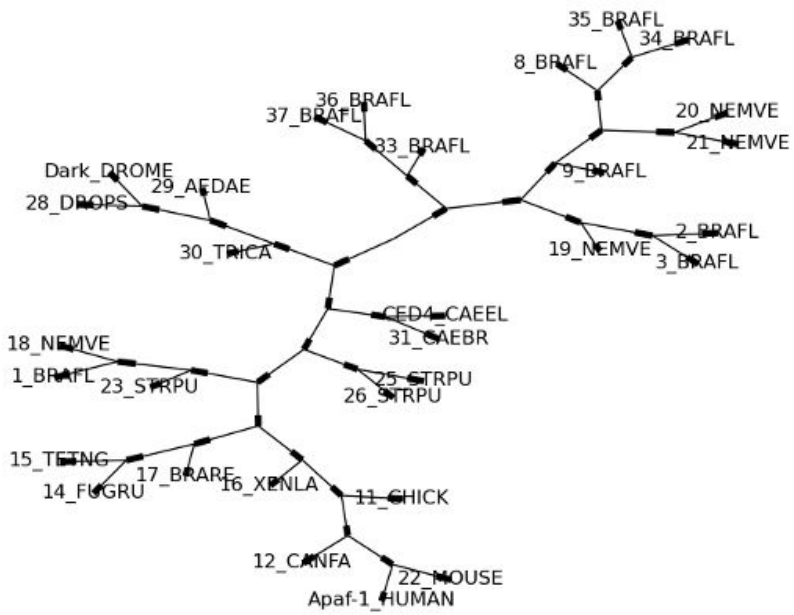
```
>>> from Bio import Entrez, SeqIO

>>> Entrez.email = "email@email.org"

>>> handle = Entrez.efetch(db="nucleotide",
                           id="186972394",
                           rettype="gb",
                           retmode="text")

>>> record = SeqIO.read(handle, "genbank")
>>> handle.close()
>>> print(record)

ID: EU490707.1
Name: EU490707
Description: Selenipedium aequinoctiale maturase K (matK) gene, partial
cds; chloroplast
Number of features: 3
/molecule_type=DNA
...
/source=chloroplast Selenipedium aequinoctiale
Seq('ATTTTTTACGAACCTGTGGAAATTTTTGGTTATGACAATAAATCTAGTTTAGTA...GAA')
```

Get the data from the database

Parse the data

http://www.ncbi.nlm.nih.gov/nuccore/186972394

Example. Phylogenetics: `Bio.Phylo`



Example. Sequence motif analysis: **`Bio.motifs`**

```
>>> from Bio.Seq import Seq
>>> from Bio import motifs

>>> instances = [Seq("TACAA"),
...              Seq("TACGC"),
...              Seq("TACAC"),
...              Seq("TACCC"),
...              Seq("AACCC"),
...              Seq("AATGC"),
...              Seq("AATGC"),
...             ]
>>> m = motifs.create(instances)

>>> print(m.counts)
        0      1      2      3      4
A:   3.00   7.00   0.00   2.00   1.00
C:   0.00   0.00   5.00   2.00   6.00
G:   0.00   0.00   0.00   3.00   0.00
T:   4.00   0.00   2.00   0.00   0.00
```

Example. Sequence motif analysis: `Bio.motifs`

```
>>> m.counts['A']
[3, 7, 0, 2, 1]

>>> m.consensus
Seq('TACGC')

>>> m.weblogo("mymotif.png")
```

- Comparing motifs, searching motifs, read motifs from MEME, …

Example. Sequence motif analysis: `Bio.motifs`

biopython / **biopython** Public

Code | Issues 419 | Pull requests 116 | Actions | Security | Insights

Weblogo returning a html page #4039

Open **arupgsh** opened this issue on Aug 17, 2022 · 18 comments

arupgsh commented on Aug 17, 2022 • edited — Contributor

With the current URL, the function is returning the HTML page itself(because of SSL redirect) and needs to be updated to "https://weblogo.threeplusone.com/create.cgi".

biopython/Bio/motifs/__init__.py
Line 506 in 2da0b26

```
506    url = "http://weblogo.threeplusone.com/create.cgi"
```

peterjc commented on Aug 23, 2022 — Member

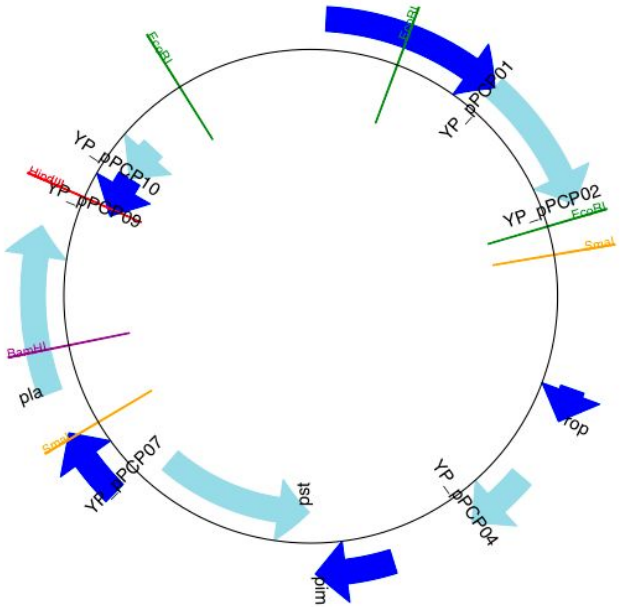
That looks like an easy fix - could you submit a pull request please?

https://github.com/biopython/biopython/issues/4039

Others:

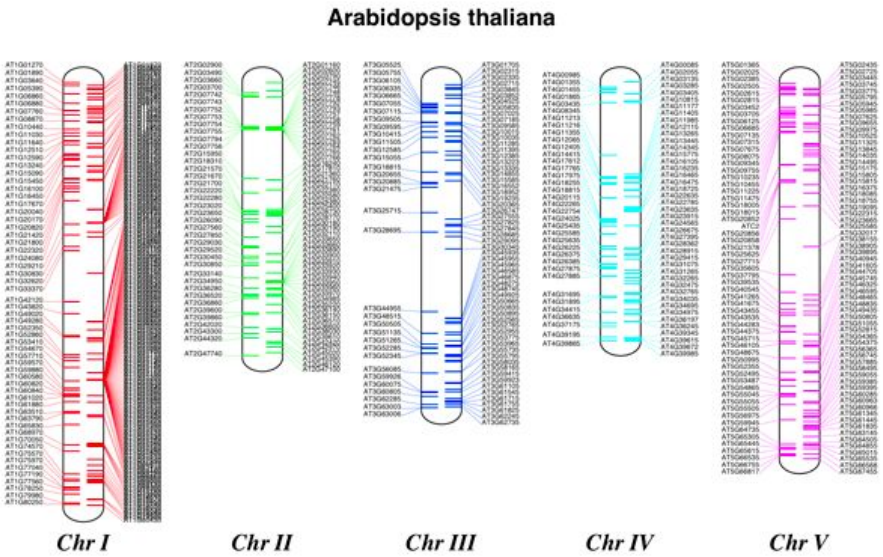
`Bio.Cluster`: clustering, cluster analysis

`Bio.Graphics.GenomeDiagram`: Genome diagrams



Others:

`Bio.Graphics.GraphicChromosome`



Others:

`Bio.Align` for seq alignments

- Summary of the alignment
- Information content
- Substitution matrices
- etc

Others:

Bio.Align

```
>>> from Bio import AlignIO
>>> alignment = AlignIO.read("PF05371_seed.sth", "stockholm")

>>> print(alignment)
SingleLetterAlphabet() alignment with 7 rows and 52 columns
AEPNAATNYATEAMDSLKTQAIDLISQTWPVVTTVVVAGLVIRL...SKA COATB_BPIKE/30-81
AEPNAATNYATEAMDSLKTQAIDLISQTWPVVTTVVVAGLVIKL...SRA Q9T0Q8_BPIKE/1-52
DGTSTATSYATEAMNSLKTQATDLIDQTWPVVTSVAVAGLAIRL...SKA COATB_BPI22/32-83
AEGDDP---AKAAFNSLQASATEYIGYAWAMVVVIVGATIGIKL...SKA COATB_BPM13/24-72
AEGDDP---AKAAFDSLQASATEYIGYAWAMVVVIVGATIGIKL...SKA COATB_BPZJ2/1-49
AEGDDP---AKAAFDSLQASATEYIGYAWAMVVVIVGATIGIKL...SKA Q9T0Q9_BPFD/1-49
FAADDATSQAKAAFDSLTAQATEMSGYAWALVVLVVGATVGIKL...SRA COATB_BPIF1/22-73



>>> for record in alignment:
...     print(record)
```

Others:

Bio.Align

Example convert between different alignment formats

```
from Bio import AlignIO

alignment = AlignIO.parse("PF05371_seed.sth", "stockholm")

count = AlignIO.write(alignment, "PF05371_seed.aln", "clustal")

print("Converted %i alignments" % count)
```

Others:

Bio.PDB: Crystal structures

from Bio.PDB import PDBList
to retrieve pdb files

l = PDBList()
l.retrieve_pdb_file("1A28")

```
>>> from Bio.PDB.PDBParser import PDBParser
>>> p = PDBParser(PERMISSIVE=1)

>>> filename = "pdb1fat.pdb"
>>> structure = p.get_structure("1fat", filename)

>>> resolution = structure.header['resolution']
>>> keywords = structure.header['keywords']
```

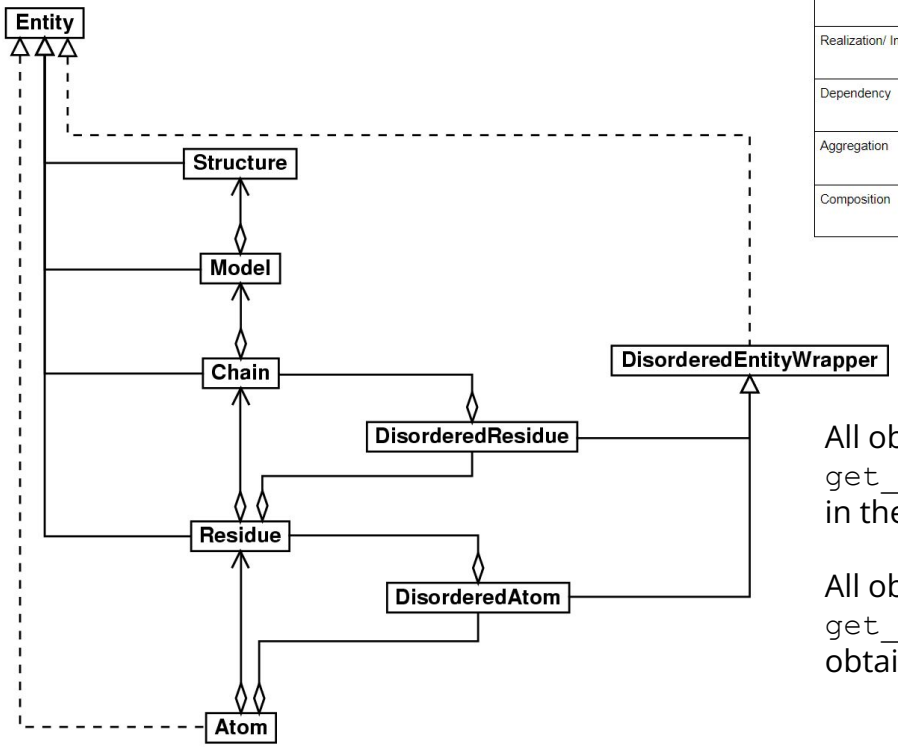
Others:

UML Diagram

Class Diagram Relationship Type	Notation
Association	
Inheritance	
Realization/ Implementation	
Dependency	
Aggregation	
Composition	

`Bio.PDB`: Crystal structures

```
for model in s:
   for chain in model:
      for residue in chain:
         for atom in residue:
            print(atom)
```



All objects have the `get_iterator()` to iterate in their inner elements

All objects have the `get_parent()` method to obtain its container object

If the structure comes from NMR we will have mutliple models, but if it ocmes from crystallography just one model.

Others:

Bio.PDB: Crystal structures

- **Structure Object**
  - Contains a number of Model objects
    - Usually one for crystal structures and several for NMR structures

```
for model_instance in structure:
    print(model_instance)

first_model = s[0]
```

Others:

`Bio.PDB`: Crystal structures

- **Residue Object**
  - Contains a number of Atom objects
  - residue_id
  - get_resname()
  - ...

Others:

Bio.PDB: Crystal structures

- **Atom Object**

```
for a in residue.get_iterator():

    print(a.get_name())
    print(a.get_occupancy())
    print(a.get_coord())
    print(a.get_vector())


CA_atom = residue["CA"]
```

Others:

Bio.PDB: Retrieving a pdb file

```
from Bio.PDB import PDBList

pdbl = PDBList()

pdbl.retrieve_pdb_file("1A02", file_format="pdb")
```